

Lộ trình 24 ngày luyện thi AZ-104 với lab C#

Oct 2, 2026 · @Khoa

Tổng quan

24 ngày phủ đủ 82 gạch đầu dòng trong [study guide AZ-104](#) (skills measured từ 17/04/2026). Số ngày chia theo trọng số; mỗi ngày làm thao tác trên portal/CLI trước, rồi một lab C# dùng Azure SDK for .NET để tự động hoá hoặc kiểm chứng đúng thứ vừa câu hình.

Domain	Trọng số	Ngày	Số gạch đầu dòng
Manage Azure identities and governance	20-25%	1-5	15
Implement and manage storage	15-20%	6-9	17
Deploy and manage Azure compute resources	20-25%	10-15	24
Implement and manage virtual networking	15-20%	16-19	13
Monitor and maintain Azure resources	10-15%	20-22	13
Practice assessment + ôn tập	—	23-24	—

Vì sao lab bằng C#, và giới hạn của nó

Đề AZ-104 hỏi bằng ngôn ngữ của admin: portal, Azure CLI, PowerShell, ARM/Bicep. Không có câu hỏi C#. Lab C# ở đây có một mục đích chiến lược: buộc bạn đi qua mô hình tài nguyên ARM (scope, resource ID, tên property) bằng code đã quen tay, vì chính các tên property đó xuất hiện lại trong ARM template, Bicep và kết quả `az ... show`. Hiểu được `properties.networkAcls.defaultAction` trong C# thì đọc template ở ngày 10 sẽ nhanh hơn nhiều.

Vì vậy mỗi ngày có ba phần: thao tác portal/CLI (đúng thứ đề thi hỏi), lab C# (hiểu sâu mô hình), và "Bãi đề thi" (các lựa chọn dễ nhầm).

Dự án xuyên suốt: AzAdminKit

Một console app .NET 8, mỗi ngày thêm một lệnh con (`dotnet run -- day03`). Cuối kỳ bạn có một bộ công cụ admin tự viết: kiểm toán quyền, dọn tài nguyên, kiểm tra cấu hình bảo mật storage, đo NSG hiệu lực.

```
AzAdminKit/
  Program.cs          // switch theo args[0] -> DayXX.RunAsync()
  Days/Day01.cs ... Days/Day22.cs
  Shared/Azure.cs     // ArmClient, GraphServiceClient dùng chung

// NuGet nên: Azure.Identity, Azure.ResourceManager, Microsoft.Graph
// Các ngày sau thêm: Azure.ResourceManager.
{Authorization,Resources,Storage,Compute,Network,
//
AppService,ContainerInstance,AppContainers,ContainerRegistry,Dns,Monitor,Reco
veryServices,...},
//  Azure.Storage.Blobs, Azure.Storage.Files.Shares, Azure.Monitor.Query
public static class Az
{
    public static readonly TokenCredential Cred = new
DefaultAzureCredential();
    public static readonly ArmClient Arm = new(Cred);

    public static async Task<ResourceGroupResource> RgAsync(string day)
    {
        var sub = await Arm.GetDefaultSubscriptionAsync();
        var data = new ResourceGroupData(AzureLocation.EastUS) { Tags = {
["course"] = "az104", ["day"] = day } };
        return (await
sub.GetResourceGroups().CreateOrUpdateAsync(WaitUntil.Completed, $"rg-az104-
{day}", data)).Value;
    }
}
```

Nhịp mỗi ngày (khoảng 2 giờ)

1. 30 phút: module Microsoft Learn tương ứng.
2. 40 phút: làm bằng portal, sau đó lặp lại bằng CLI hoặc PowerShell (đề thi hay cho đoạn lệnh và hỏi thiếu tham số nào).
3. 40 phút: lab C# của ngày.

4. 10 phút: tự viết 3 câu hỏi "khi nào chọn X thay vì Y", rồi xoá resource group trong ngày.

Chiến lược credit \$200

Nếu xoá resource group cuối ngày, toàn bộ lịch tổn khoảng \$40–70. Credit hết hạn sau 30 ngày dù dùng hay không, nên phần dư cứ dùng để làm lại lab đắt (App Gateway không có trong đề, nhưng Bastion, VMSS, Site Recovery thì có). Đặt Budget cảnh báo \$50 / \$100 / \$150 ngay ngày 1, bật auto-shutdown cho mọi VM, và bật dùng thử Entra ID P2 (miễn phí 30 ngày) cho dynamic group và SSPR nâng cao.

Ngày 1–5: Manage Azure identities and governance (20–25%)

Năm ngày cho 15 gạch đầu dòng. Hai ngày đầu dùng Microsoft Graph SDK (Entra ID), ba ngày sau dùng Azure Resource Manager SDK. Chạy lab bằng tài khoản Global Administrator của tenant free account; nếu Graph báo thiếu quyền, đăng nhập lại bằng `az login --scope https://graph.microsoft.com//.default`.

Ngày 1 — Tạo và quản lý user, group

Bám bullet: tạo user và group; quản lý thuộc tính user và group.

Portal/CLI: tạo 3 user (một bằng bulk CSV), một security group kiểu Assigned, một kiểu Dynamic theo `department`; xoá rồi khôi phục một user trong mục Deleted users.

```
az ad user create --display-name "Minh Tran" --user-principal-name
minh.tran@<domain> --password '<P@ss>' --force-change-password-next-sign-in
az ad group create --display-name sg-engineering --mail-nickname sg-
engineering
az ad group member add --group sg-engineering --member-id $(az ad user show -
-id minh.tran@<domain> --query id -o tsv)
```

Lab C# (Day01):

```
var graph = new GraphServiceClient(Az.Cred,
["https://graph.microsoft.com/.default"]);
var domain = (await graph.Domains.GetAsync())!.Value!.First(d => d.IsDefault
== true).Id;

var user = await graph.Users.PostAsync(new User
{
    AccountEnabled = true, DisplayName = "Lan Nguyen", MailNickname =
```

```

"lan.nguyen",
    UserPrincipalName = $"lan.nguyen@{domain}", Department = "Engineering",
    UsageLocation = "VN",
    PasswordProfile = new PasswordProfile { Password = "Az104!" +
    Guid.NewGuid().ToString("N")[..8], ForceChangePasswordNextSignIn = true }
});

// Group kiểu Assigned + thêm thành viên
var sg = await graph.Groups.PostAsync(new Group { DisplayName = "sg-lab-
assigned", MailNickname = "sg-lab-assigned", MailEnabled = false,
SecurityEnabled = true });
await graph.Groups[sg!.Id].Members.Ref.PostAsync(new ReferenceCreate {
OdataId = $"https://graph.microsoft.com/v1.0/directoryObjects/{user!.Id}" });

// Group kiểu Dynamic (cần Entra ID P1/P2)
await graph.Groups.PostAsync(new Group
{
    DisplayName = "dyn-engineering", MailNickname = "dyn-engineering",
    MailEnabled = false, SecurityEnabled = true,
    GroupTypes = ["DynamicMembership"], MembershipRule = "(user.department -
eq \"Engineering\")", MembershipRuleProcessingState = "On"
});

// Quản lý thuộc tính
await graph.Users[user.Id].PatchAsync(new User { JobTitle = "Cloud Admin",
OfficeLocation = "HCMC" });
var members = await graph.Groups[sg.Id].Members.GetAsync();
Console.WriteLine($"{sg.DisplayName}: {members!.Value!.Count} members");

```

Bắt đề thi: Security group dùng để phân quyền, Microsoft 365 group dùng cho cộng tác; group Dynamic không cho thêm thành viên thủ công và cần license P1; đổi Assigned sang Dynamic xóa toàn bộ thành viên hiện có; user bị xóa nằm trong Deleted users 30 ngày.

Chi phí: \$0.

Ngày 2 — License, external user, SSPR

Bám bullet: quản lý license trong Entra ID; quản lý external user; cấu hình self-service password reset.

Portal: bật dùng thử Entra ID P2; gán license cho group `sg-lab-assigned` (group-based licensing); mời một email cá nhân làm guest; cấu hình SSPR cho group Selected, 2 phương thức, bắt buộc đăng ký khi sign-in; chỉnh External collaboration settings để chỉ admin

được mời guest.

Lab C# (Day02):

```
// License: user phải có UsageLocation trước khi gán
var skus = await graph.SubscribedSkus.GetAsync();
foreach (var s in skus!.Value!)
    Console.WriteLine($"{s.SkuPartNumber}:
{s.ConsumedUnits}/{s.PrepaidUnits!.Enabled}");
var p2 = skus.Value!.First(s => s.SkuPartNumber == "AAD_PREMIUM_P2");
await graph.Users[userId].AssignLicense.PostAsync(new
AssignLicensePostRequestBody
{
    AddLicenses = [new AssignedLicense { SkuId = p2.SkuId }], RemoveLicenses
= []
});

// External user (B2B)
var inv = await graph.Invitations.PostAsync(new Invitation
{
    InvitedUserEmailAddress = "<email-ca-nhan>", InviteRedirectUrl =
"https://myapps.microsoft.com", SendInvitationMessage = true
});
Console.WriteLine($"Guest {inv!.InvitedUser!.Id}, redeem:
{inv.InviteRedeemUrl}");
var guests = await graph.Users.GetAsync(r => r.QueryParameters.Filter =
"userType eq 'Guest'");

// SSPR: kiểm tra ai đã đăng ký phương thức reset
var reg = await
graph.Reports.AuthenticationMethods.UserRegistrationDetails.GetAsync();
foreach (var r in reg!.Value!)
    Console.WriteLine($"{r.UserPrincipalName}: ssprEnabled={r.IsSsprEnabled}
registered={r.IsSsprRegistered}");
```

Bây đề thi: thiếu Usage location thì không gán được license; group-based licensing cần P1; tài khoản admin luôn có SSPR với chính sách hai phương thức, không tắt được; password writeback về AD on-prem cần Entra Connect; guest có `userType = Guest` và chịu External collaboration settings.

Chi phí: \$0.

Ngày 3 — Azure RBAC

Bám bullet: quản lý built-in role; gán role ở các scope khác nhau; diễn giải access assignment.

Portal/CLI: gán Reader cho group ở subscription, Contributor cho user ở resource group, Storage Blob Data Reader ở một storage account; dùng Check access để xem quyền hiệu lực.

```
az role definition list --name "Virtual Machine Contributor" --query "[].permissions[].actions"
az role assignment create --assignee <groupId> --role Reader --scope /subscriptions/<subId>
az role assignment list --assignee <userUpn> --all --include-inherited -o table
```

Lab C# (Day03) — AccessAuditor: liệt kê mọi assignment tác động lên một resource group, đánh dấu cái nào kế thừa từ scope cha.

```
var sub = await Az.Arm.GetDefaultSubscriptionAsync();
var rg = await Az.RgAsync("d03");

// Tra role built-in theo tên
AuthorizationRoleDefinitionResource? reader = null;
await foreach (var rd in
sub.GetAuthorizationRoleDefinitions().GetAllAsync(filter: "roleName eq 'Reader'"))
    reader = rd;

// Gán ở scope resource group
await rg.GetRoleAssignments().CreateOrUpdateAsync(WaitUntil.Completed,
Guid.NewGuid().ToString(),
    new RoleAssignmentCreateOrUpdateContent(reader!.Id,
Guid.Parse(principalId)) { PrincipalType = RoleManagementPrincipalType.Group
});

// atScope(): assignment tại scope này và các scope cha
var roleNames = new Dictionary<string, string>();
await foreach (var ra in rg.GetRoleAssignments().GetAllAsync(filter:
"atScope()"))
{
    var defId = ra.Data.RoleDefinitionId.ToString();
    if (!roleNames.TryGetValue(defId, out var name))
```

```

        roleNames[defId] = name = (await
Az.Arm.GetAuthorizationRoleDefinitionResource(ra.Data.RoleDefinitionId).GetAs
ync()).Value.Data.RoleName;
        var inherited = !ra.Data.Scope.Equals(rg.Id.ToString(),
StringComparison.OrdinalIgnoreCase);
        Console.WriteLine($"{ra.Data.PrincipalType,-16} {ra.Data.PrincipalId}
{name,-28} {(inherited ? "INHERITED from " + ra.Data.Scope : "direct")}");
    }

```

Bẫy đề thi: quyền RBAC cộng dồn, quyền rộng nhất thắng (trừ deny assignment); Owner = Contributor + quyền gán role; User Access Administrator chỉ quản lý quyền, không quản lý tài nguyên; Entra role (Global Admin, User Admin) khác hoàn toàn Azure role; role data plane như `Storage Blob Data Reader` khác role control plane như `Reader`.

Chi phí: \$0.

Ngày 4 — Azure Policy, resource lock, tag

Bám bullet: triển khai và quản lý Azure Policy; cấu hình resource lock; áp dụng và quản lý tag.

Portal: gán policy "Allowed locations" ở resource group, initiative có "Require a tag on resource groups", policy Modify "Inherit a tag from the resource group" kèm remediation task; đặt lock `CanNotDelete` và `ReadOnly` rồi thử thao tác.

Lab C# (Day04):

```

var rg = await Az.RgAsync("d04");
var tenant = Az.Arm.GetTenants().First();

// Tìm built-in definition theo tên thay vì hard-code GUID
ResourceIdentifier? allowedId = null;
await foreach (var d in
tenant.GetTenantPolicyDefinitions().GetAllAsync(filter: "policyType eq
'BuiltIn'))
    if (d.Data.DisplayName == "Allowed locations") { allowedId = d.Id; break;
}

await rg.GetPolicyAssignments().CreateOrUpdateAsync(WaitUntil.Completed,
"allowed-locations", new PolicyAssignmentData
{
    PolicyDefinitionId = allowedId, DisplayName = "Only East US",
    Parameters = { ["listOfAllowedLocations"] = new ArmPolicyParameterValue {

```

```

Value = BinaryData.FromObjectAsJson(new[] { "eastus" }) } }
});

// Thu'vi phạm: tạo storage ở West Europe
try
{
    await rg.GetStorageAccounts().CreateOrUpdateAsync(WaitUntil.Completed,
        $"stdeny{Random.Shared.Next(99999)}",
        new StorageAccountCreateOrUpdateContent(new
            StorageSku(StorageSkuName.StandardLrs), StorageKind.StorageV2,
            AzureLocation.WestEurope));
}
catch (RequestFailedException ex) { Console.WriteLine($"Bị chặn:
{ex.ErrorCode}"); } // RequestDisallowedByPolicy

// Lock và tag
await rg.GetManagementLocks().CreateOrUpdateAsync(WaitUntil.Completed, "no-
delete",
    new ManagementLockData(ManagementLockLevel.CanNotDelete) { Notes = "AZ-
104 lab" });
await rg.AddTagAsync("costCenter", "az104");

// Báo cáo: tài nguyên thiếu tag costCenter
var sub = await Az.Arm.GetDefaultSubscriptionAsync();
await foreach (var r in sub.GetGenericResourcesAsync())
    if (!r.Data.Tags.ContainsKey("costCenter")) Console.WriteLine($"Thiếu
tag: {r.Id}");

```

Bẫy đề thi: tag không tự kế thừa từ resource group (cần policy Modify); lock thì kế thừa xuống tài nguyên con; `ReadOnly` chặn cả thao tác POST như list storage keys và có thể làm hỏng một số thao tác tưởng chỉ đọc; policy Modify và DeployIfNotExists cần managed identity, và tài nguyên đã tồn tại chỉ được sửa qua remediation task; muốn xóa resource group có lock phải gỡ lock trước.

Chi phí: \$0. Gỡ lock trước khi xóa resource group.

Ngày 5 — Resource group, subscription, chi phí, management group

Bám bullet: quản lý resource group; quản lý subscription; quản lý chi phí bằng alert, budget và Azure Advisor; cấu hình management group.

Portal/CLI: tạo budget có action group, xem Cost analysis theo tag, đọc khuyến nghị Advisor nhóm Cost; tạo cây management group `mg-az104 > mg-lab` và đưa subscription vào `mg-lab`.

```
az consumption budget create --budget-name az104-monthly --amount 150 --time-grain Monthly \
  --start-date 2026-10-01 --end-date 2026-12-31 --category cost
az advisor recommendation list --category Cost -o table
az account management-group create -n mg-az104 --display-name "AZ-104 Lab"
```

Lab C# (Day05): di chuyển tài nguyên giữa resource group có kiểm tra trước, kiểm tra quota vCPU, và dựng management group bằng SDK.

```
var sub = await Az.Arm.GetDefaultSubscriptionAsync();
var src = await Az.RgAsync("d05-src");
var dst = await Az.RgAsync("d05-dst");
var st = (await
src.GetStorageAccounts().CreateOrUpdateAsync(WaitUntil.Completed,
$"stmove{Random.Shared.Next(99999)}",
    new StorageAccountCreateOrUpdateContent(new
StorageSku(StorageSkuName.StandardLrs), StorageKind.StorageV2,
AzureLocation.EastUS))).Value;

var move = new ResourcesMoveContent { TargetResourceGroupId = dst.Id,
Resources = { st.Id } };
await src.ValidateMoveResourcesAsync(WaitUntil.Completed, move); // ném lỗi
nếu không di chuyển được
await src.MoveResourcesAsync(WaitUntil.Completed, move);

// Quota vCPU (free account giới hạn thấp, cần biết trước khi làm VMSS)
await foreach (var u in sub.GetUsagesAsync(AzureLocation.EastUS))
    if (u.Name.Value.Contains("cores", StringComparison.OrdinalIgnoreCase))
        Console.WriteLine($"{u.Name.LocalizedValue}:
{u.CurrentValue}/{u.Limit}");

// Management group
var tenant = Az.Arm.GetTenants().First();
var mg = (await
tenant.GetManagementGroups().CreateOrUpdateAsync(WaitUntil.Completed, "mg-
az104-lab",
    new ManagementGroupCreateOrUpdateContent { DisplayName = "AZ-104 Lab"
```

```

    })).Value;
    await
    mg.GetManagementGroupSubscriptions().CreateOrUpdateAsync(WaitUntil.Completed,
    sub.Data.SubscriptionId);

```

Bẫy đề thi: location của resource group chỉ chứa metadata, tài nguyên bên trong có thể ở region khác; di chuyển tài nguyên không đổi region; xoá resource group xoá mọi thứ bên trong; budget chỉ cảnh báo chứ không tự dừng tài nguyên; cây management group tối đa 6 cấp (không tính root), mỗi subscription chỉ thuộc một management group; policy và RBAC gán ở management group áp xuống mọi subscription bên dưới.

Chi phí: ~\$0.1.

Ngày 6–9: Implement and manage storage (15–20%)

Bốn ngày cho 17 gạch đầu dòng, đi từ account đến bảo mật truy cập, rồi Blob và Azure Files. Các lab C# dùng cả hai tầng: ARM SDK (`Azure.ResourceManager.Storage`, control plane) và Storage SDK (`Azure.Storage.Blobs`, `Azure.Storage.Files.Shares`, data plane). Phân biệt hai tầng này chính là chỗ đề hay gài bẫy.

Ngày 6 — Tạo và cấu hình storage account, redundancy, encryption

Bám bullet: tạo và cấu hình storage account; cấu hình redundancy; cấu hình encryption cho storage account.

Portal/CLI: tạo account StorageV2 LRS, sau đó đổi sang GRS rồi RA-GRS; xem tab Encryption (Microsoft-managed key, tùy chọn customer-managed key, infrastructure encryption).

```

az storage account create -g rg-az104-d06 -n st104$RANDOM --sku Standard_LRS
--kind StorageV2 \
  --min-tls-version TLS1_2 --allow-blob-public-access false --require-
infrastructure-encryption
az storage account update -g rg-az104-d06 -n <st> --sku Standard_RAGRS

```

Lab C# (Day06) — StorageAuditor: tạo account bằng SDK với cấu hình an toàn, đổi redundancy, rồi quét mọi account trong subscription.

```

var rg = await Az.RgAsync("d06");
var content = new StorageAccountCreateOrUpdateContent(new
StorageSku(StorageSkuName.StandardLrs), StorageKind.StorageV2,
AzureLocation.EastUS)

```

```
{
    AccessTier = StorageAccountAccessTier.Hot,
    MinimumTlsVersion = StorageMinimumTlsVersion.Tls1_2,
    AllowBlobPublicAccess = false,
    EnableHttpsTrafficOnly = true,
    Encryption = new StorageAccountEncryption { KeySource =
StorageAccountKeySource.Storage, RequireInfrastructureEncryption = true }
};
var st = (await
rg.GetStorageAccounts().CreateOrUpdateAsync(WaitUntil.Completed,
$"st104{Random.Shared.Next(99999)}", content)).Value;

// Đổi redundancy LRS -> GRS (thao tác control plane, không ảnh hưởng dữ
liệu)
await st.UpdateAsync(new StorageAccountPatch { Sku = new
StorageSku(StorageSkuName.StandardGrs) });

// Kiểm toán
var sub = await Az.Arm.GetDefaultSubscriptionAsync();
await foreach (var a in sub.GetStorageAccountsAsync())
    Console.WriteLine($"{{a.Data.Name,-24}} {{a.Data.Sku.Name,-16}} tls=
{{a.Data.MinimumTlsVersion}} " +
        $"publicBlob={{a.Data.AllowBlobPublicAccess}} infraEnc=
{{a.Data.Encryption?.RequireInfrastructureEncryption}} " +
        $"secondary={{a.Data.SecondaryLocation}}");
```

Đọc template (Bicep): không chạy, chỉ đọc rồi trả lời.

```
param location string = resourceGroup().location
param namePrefix string = 'stlab'
@allowed(['Standard_LRS', 'Standard_ZRS', 'Standard_GRS'])
param skuName string = 'Standard_LRS'

resource st 'Microsoft.Storage/storageAccounts@2023-05-01' = {
    name: '${namePrefix}${uniqueString(resourceGroup().id)}'
    location: location
    sku: { name: skuName }
    kind: 'StorageV2'
    properties: {
        accessTier: 'Cool'
        allowBlobPublicAccess: true
        minimumTlsVersion: 'TLS1_0'
        networkAcls: {
```

```

        defaultAction: 'Deny'
        bypass: 'AzureServices'
        ipRules: [ { value: '203.0.113.0/24', action: 'Allow' } ]
    }
}
}

```

1. Deploy với `--parameters skuName=Standard_RAGRS` thì chuyện gì xảy ra?
2. Cần sửa hai dòng nào để đạt chuẩn: không cho truy cập ẩn danh và TLS tối thiểu 1.2?
3. Người dùng ở IP `198.51.100.25` nhận lỗi 403 khi đọc blob. Vì sao, sửa thể nào?
4. Blob mới upload mặc định nằm ở tier nào?

Đáp án: (1) Deploy thất bại ngay ở bước validate vì giá trị không nằm trong `@allowed`. (2) `allowBlobPublicAccess: false` và `minimumTlsVersion: 'TLS1_2'`. (3) `defaultAction: 'Deny'` và IP đó không nằm trong `ipRules`; thêm một rule cho IP hoặc dải của người dùng. (4) Cool, theo `accessTier` của account.

Bây đề thi: thuộc lòng bảng LRS (3 bản, 1 datacenter), ZRS (3 zone), GRS/RA-GRS (thêm region phụ, RA cho phép đọc region phụ), GZRS/RA-GZRS; Premium chỉ hỗ trợ LRS/ZRS; chuyển sang hoặc ra khỏi ZRS là chuyển đổi đặc biệt, không chỉ đổi SKU; mã hoá at rest luôn bật; customer-managed key cần Key Vault có soft delete và purge protection; infrastructure encryption chỉ bật được lúc tạo account.

Chi phí: vài cent.

Ngày 7 — Firewall, SAS, stored access policy, access key

Bám bullet: câu hình firewall và virtual network cho Azure Storage; tạo và dùng SAS token; câu hình stored access policy; quản lý access key.

Portal: Networking chuyển sang "Enabled from selected virtual networks and IP addresses", thêm IP của bạn và tick ngoại lệ trusted Microsoft services; tạo SAS ở mức account và mức blob; tạo stored access policy trên container; xoay key1/key2.

Lab C# (Day07):

```

// Firewall (control plane)
await st.UpdateAsync(new StorageAccountPatch
{
    NetworkRuleSet = new
StorageAccountNetworkRuleSet(StorageNetworkDefaultAction.Deny)
{

```

```

        Bypass = StorageNetworkBypass.AzureServices,
        IPRules = { new StorageAccountIPRule(myPublicIp) }
    }
});

// Access key: liệt kê và xoay vòng key2
await foreach (var k in st.GetKeysAsync()) Console.WriteLine($"{k.KeyName}:
...{k.Value[^6..]}");
await foreach (var _ in st.RegenerateKeyAsync(new
StorageAccountRegenerateKeyContent("key2"))) { }

// Service SAS ký bằng account key
var keyCred = new StorageSharedKeyCredential(st.Data.Name, key1);
var container = new BlobContainerClient(new
Uri($"https://{st.Data.Name}.blob.core.windows.net/reports"), keyCred);
await container.CreateIfNotExistsAsync();
var sas =
container.GetBlobClient("q3.csv").GenerateSasUri(BlobSasPermissions.Read,
DateTimeOffset.UtcNow.AddHours(1));

// Stored access policy: SAS tham chiếu policy, thu hồi bằng cách xoá policy
await container.SetAccessPolicyAsync(permissions: [ new BlobSignedIdentifier
{
    Id = "read-7d",
    AccessPolicy = new BlobAccessPolicy { PolicyExpiresOn =
DateTimeOffset.UtcNow.AddDays(7), Permissions = "r" }
}]);
var policySas = container.GenerateSasUri(new BlobSasBuilder {
BlobContainerName = "reports", Identifier = "read-7d" });

// User delegation SAS: ký bằng Entra ID, không dùng account key
var svc = new BlobServiceClient(new
Uri($"https://{st.Data.Name}.blob.core.windows.net"), Az.Cred);
var udk = (await svc.GetUserDelegationKeyAsync(DateTimeOffset.UtcNow,
DateTimeOffset.UtcNow.AddHours(1))).Value;
var b = new BlobSasBuilder(BlobSasPermissions.Read,
DateTimeOffset.UtcNow.AddHours(1)) { BlobContainerName = "reports", BlobName
= "q3.csv" };
var udSas = new BlobUriBuilder(container.GetBlobClient("q3.csv").Uri) { Sas =
b.ToSasQueryParameters(udk, st.Data.Name) }.ToUri();

```

Thử truy cập từ IP khác (điện thoại 4G) để thấy firewall chặn, rồi xoá policy `read-7d` để thấy SAS dựa trên policy chết ngay.

Bắt đề thi: account key cho toàn quyền, nên xoay vòng theo kiểu hai key; xoay key làm mất hiệu lực mọi SAS ký bằng key đó; user delegation SAS chỉ cho Blob/Data Lake, sống tối đa 7 ngày, là lựa chọn an toàn nhất; mỗi container tối đa 5 stored access policy; firewall **Deny** cũng chặn chính bạn xem dữ liệu trên portal nếu chưa thêm IP.

Chi phí: vài cent.

Ngày 8 — Blob: container, tier, soft delete, versioning, lifecycle, object replication, AzCopy

Bám bullet: tạo và cấu hình container trong Blob Storage; cấu hình storage tier; cấu hình soft delete cho blob và container; cấu hình blob lifecycle management; cấu hình blob versioning; cấu hình object replication; quản lý dữ liệu bằng Storage Explorer và AzCopy.

Portal/CLI: bật soft delete blob + container 7 ngày, versioning, change feed; tạo lifecycle rule; tạo account thứ hai và object replication từ container **src** sang **dst**; copy dữ liệu bằng AzCopy và duyệt bằng Storage Explorer.

```
az storage account blob-service-properties update -g $RG --account-name <st> \
  --enable-delete-retention true --delete-retention-days 7 \
  --enable-container-delete-retention true --container-delete-retention-days 7 \
  --enable-versioning true --enable-change-feed true
az storage account management-policy create -g $RG --account-name <st> --
policy @lifecycle.json
az storage account or-policy create -g $RG -n <st-dst> --source-account <st-
src> --destination-account <st-dst> \
  --source-container src --destination-container dst --min-creation-time
2026-01-01T00:00:00Z
azcopy login && azcopy copy ./data "https://<st>.blob.core.windows.net/src" -
-recursive
azcopy sync ./data "https://<st>.blob.core.windows.net/src" --delete-
destination true
```

```
{ "rules": [ { "name": "logs-aging", "enabled": true, "type": "Lifecycle",
  "definition": { "filters": { "blobTypes": ["blockBlob"], "prefixMatch":
["src/logs/"] },
    "actions": {
      "baseBlob": { "tierToCool": { "daysAfterModificationGreaterThan": 30 },
        "tierToArchive": { "daysAfterModificationGreaterThan": 90
},
      "delete": { "daysAfterModificationGreaterThan": 365 } },
    "version": { "delete": { "daysAfterCreationGreaterThan": 30 } } } } ]
}
```

Lab C# (Day08): sinh version, xoá rồi khôi phục, đổi tier, và in trạng thái mọi phiên bản.

```
var c = svc.GetBlobContainerClient("src");
await c.CreateIfNotExistsAsync(PublicAccessType.None);
var blob = c.GetBlobClient("logs/app.log");
for (var i = 1; i <= 3; i++)
    await blob.UploadAsync(BinaryData.FromString($"version {i}"), overwrite:
true); // mỗi lần ghi đè tạo 1 version

await blob.DeleteAsync(); // soft delete
await blob.UndeleteAsync(); // khôi phục trong thời gian retention
await blob.SetAccessTierAsync(AccessTier.Cool);

await foreach (var item in c.GetBlobsAsync(states: BlobStates.Version |
BlobStates.Deleted, prefix: "logs/"))
    Console.WriteLine($"{item.Name} version={item.VersionId} current=
{item.IsLatestVersion} deleted={item.Deleted} tier=
{item.Properties.AccessTier}");

// Đưa một version cũ trở lại làm bản hiện tại
var oldest = /* VersionId của version 1 lấy từ vòng lặp trên */ "";
await blob.StartCopyFromUriAsync(blob.WithVersion(oldest).Uri);
```

Bây đề thi: Archive là offline, phải rehydrate (Standard tới 15 giờ, High priority nhanh hơn); phí xoá sớm: Cool 30 ngày, Cold 90 ngày, Archive 180 ngày; tier mặc định của account chỉ có Hot/Cool/Cold, không có Archive; lifecycle chạy khoảng mỗi ngày một lần, thay đổi có thể mất tới 24 giờ; object replication cần versioning ở cả hai account và change feed ở account nguồn; soft delete container và soft delete blob là hai thiết lập riêng; `azcopy sync` so sánh theo thời gian sửa đổi, `copy` thì không.

Chi phí: vài cent.

Ngày 9 — Azure Files: share, snapshot, soft delete, identity-based access

Bám bullet: tạo và cấu hình file share; cấu hình snapshot và soft delete cho Azure Files; cấu hình identity-based access cho Azure Files.

Portal/CLI: tạo share có quota và tier; bật soft delete cho share; tạo snapshot; mount share trên một VM Linux B1s (nhà mạng thường chặn cổng 445 nên mount từ máy nhà dễ thất bại); xem các lựa chọn identity source (AD DS, Entra Domain Services, Entra Kerberos) và đặt default share-level permission.

```
az storage share-rm create -g $RG --storage-account <st> -n team --quota 100
--access-tier TransactionOptimized
az storage account file-service-properties update -g $RG --account-name <st>
--enable-delete-retention true --delete-retention-days 7
az storage share-rm snapshot -g $RG --storage-account <st> -n team
sudo mount -t cifs //<st>.file.core.windows.net/team /mnt/team -o
vers=3.1.1,username=<st>,password=<key>,serverino
```

Lab C# (Day09): tạo share, ghi file, chụp snapshot, sửa file, khôi phục file từ snapshot; gán role SMB ở scope share.

```

var share = new ShareClient(connString, "team");
await share.CreateIfNotExistsAsync(new ShareCreateOptions { QuotaInGB = 100
});
var file = share.GetRootDirectoryClient().GetFileClient("policy.txt");
await using (var s = new MemoryStream(Encoding.UTF8.GetBytes("v1"))) { await
file.CreateAsync(s.Length); await file.UploadAsync(s); }

var snap = (await share.CreateSnapshotAsync()).Value.Snapshot;      //
snapshot cấp share, chỉ đọc
await using (var s = new MemoryStream(Encoding.UTF8.GetBytes("v2-broken"))) {
await file.CreateAsync(s.Length); await file.UploadAsync(s); }

var fromSnap =
share.WithSnapshot(snap).GetRootDirectoryClient().GetFileClient("policy.txt")
;
await file.StartCopyAsync(fromSnap.Uri);                          // khôì
phục bản v1
Console.WriteLine((await
file.DownloadContentAsync()).Value.Content.ToString());

// Role SMB ở scope share (control plane)
var shareScope = new ResourceIdentifier($"
{st.Id}/fileServices/default/fileshares/team");
var roleDef = new
ResourceIdentifier($" /subscriptions/{subId}/providers/Microsoft.Authorization
/roleDefinitions/<id-cua-Storage-File-Data-SMB-Share-Contributor>");
await
Az.Arm.GetRoleAssignments(shareScope).CreateOrUpdateAsync(WaitUntil.Completed
, Guid.NewGuid().ToString(),
    new RoleAssignmentCreateOrUpdateContent(roleDef, Guid.Parse(groupId)));

```

Bắt đề thi: soft delete của Azure Files bảo vệ cả share, không phải từng file; snapshot là cấp share, tối đa 200 snapshot; quyền chia hai lớp: share-level (RBAC: SMB Share Reader / Contributor / Elevated Contributor) và directory/file-level (ACL kiểu NTFS); identity-based access chỉ chọn một nguồn AD tại một thời điểm; SMB dùng cổng TCP 445.

Chi phí: ~\$0.3 (VM B1s vài giờ).

Ngày 10–15: Deploy and manage Azure compute resources (20–25%)

Sáu ngày cho 24 gạch đầu dòng, domain nhiều gạch đầu dòng nhất. Free account thường chỉ có vài vCPU mỗi region (đã kiểm tra ở ngày 5), nên mọi VM dùng B1s/B2s và bật auto-shutdown.

Ngày 10 — ARM template và Bicep

Bám bullet: diễn giải ARM template hoặc Bicep; sửa ARM template có sẵn; sửa Bicep có sẵn; deploy bằng ARM template hoặc Bicep; export deployment thành ARM template hoặc chuyển ARM template sang Bicep.

Portal/CLI: tạo vài tài nguyên bằng portal, export template của resource group, decompile sang Bicep, sửa rồi deploy lại với what-if.

```
az group export -g rg-az104-d10 > exported.json
az bicep decompile --file exported.json          # sinh exported.bicep (best effort, cần dọn lại)
az deployment group what-if -g rg-az104-d10 --template-file main.bicep --parameters env=dev
az deployment group create -g rg-az104-d10 --template-file main.bicep --parameters env=dev --mode Incremental
```

Đoạn Bicep để luyện đọc và sửa (thêm một tham số `skuName`, một vòng lặp tạo 2 container):

```
@allowed(['dev', 'prod'])
param env string
param location string = resourceGroup().location
var stName = 'st${env}${uniqueString(resourceGroup().id)}'

resource st 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: stName
  location: location
  sku: { name: env == 'prod' ? 'Standard_GRS' : 'Standard_LRS' }
  kind: 'StorageV2'
  tags: { env: env }
  properties: { minimumTlsVersion: 'TLS1_2', allowBlobPublicAccess: false }
}

resource blobSvc 'Microsoft.Storage/storageAccounts/blobServices@2023-05-01'
```

```
= { parent: st, name: 'default' }
resource containers
'Microsoft.Storage/storageAccounts/blobServices/containers@2023-05-01' = [for
c in ['raw', 'curated']: {
  parent: blobSvc
  name: c
}]
output blobEndpoint string = st.properties.primaryEndpoints.blob
```

Cùng nội dung viết bằng ARM JSON, để luyện chuyển qua lại giữa hai dạng (đề thi dùng cả hai):

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-
01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "env": { "type": "string", "allowedValues": [ "dev", "prod" ] },
    "location": { "type": "string", "defaultValue": "[
resourceGroup().location]" }
  },
  "variables": {
    "stName": "[concat('st', parameters('env'),
uniqueString(resourceGroup().id))]",
    "containers": [ "raw", "curated" ]
  },
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "apiVersion": "2023-05-01",
      "name": "[variables('stName')]",
      "location": "[parameters('location')]",
      "sku": { "name": "[if(equals(parameters('env'), 'prod'),
'Standard_GRS', 'Standard_LRS')]" },
      "kind": "StorageV2",
      "tags": { "env": "[parameters('env')]" },
      "properties": { "minimumTlsVersion": "TLS1_2", "allowBlobPublicAccess":
false }
    },
    {
      "type": "Microsoft.Storage/storageAccounts/blobServices/containers",
      "apiVersion": "2023-05-01",
      "name": "[format('{0}/default/{1}', variables('stName'),
```

```
variables('containers')[copyIndex()]]",
    "copy": { "name": "containerLoop", "count": "[length(variables('containers'))]" },
    "dependsOn": [ "[resourceId('Microsoft.Storage/storageAccounts', variables('stName'))]" ]
  },
],
"outputs": {
  "blobEndpoint": { "type": "string", "value": "[reference(variables('stName')).primaryEndpoints.blob]" }
}
```

Các cặp tương ứng cần nhận ra ngay khi đọc:

Bicep	ARM JSON
<code>param env string + @allowed</code>	<code>parameters.env</code> với <code>allowedValues</code>
<code>var stName = 'st\${env}...'</code>	<code>variables.stName</code> với <code>concat()</code> hoặc <code>format()</code>
<code>env == 'prod' ? 'A' : 'B'</code>	<code>if(equals(parameters('env'), 'prod'), 'A', 'B')</code>
<code>[for c in [...]: {...}]</code>	<code>copy + copyIndex()</code>
<code>parent: blobSvc</code>	tên ghép bằng dấu <code>/</code> (<code>stName/default/raw</code>)
phụ thuộc tự suy ra từ symbol	<code>dependsOn + resourceId()</code> khai báo tay
<code>st.properties.primaryEndpoints.blob</code>	<code>reference(...).primaryEndpoints.blob</code>

Lab C# (Day10): deploy template JSON từ code, chạy what-if, rồi export lại resource group.

```

var rg = await Az.RgAsync("d10");
var template = BinaryData.FromString(await
File.ReadAllTextAsync("main.json")); // az bicep build --file main.bicep
var parameters = BinaryData.FromObjectAsJson(new { env = new { value = "dev"
} });

var dep = (await
rg.GetArmDeployments().CreateOrUpdateAsync(WaitUntil.Completed, "d10-main",
    new ArmDeploymentContent(new
ArmDeploymentProperties(ArmDeploymentMode.Incremental) { Template = template,
Parameters = parameters }))).Value;
Console.WriteLine($"{dep.Data.Properties.ProvisioningState}, outputs:
{dep.Data.Properties.Outputs}");

var whatIf = await dep.WhatIfAsync(WaitUntil.Completed, new
ArmDeploymentWhatIfContent(
    new ArmDeploymentWhatIfProperties(ArmDeploymentMode.Complete) { Template
= template, Parameters = parameters }));
foreach (var ch in whatIf.Value.Changes) Console.WriteLine($"
{ch.ChangeType,-10} {ch.ResourceId}"); // Complete mode sẽ báo Delete

var exported = await rg.ExportTemplateAsync(WaitUntil.Completed, new
ExportTemplate { Resources = { "*" } });
await File.WriteAllTextAsync("exported.json",
exported.Value.Template.ToString());

```

Bẫy đề thi: Incremental giữ tài nguyên không có trong template, Complete xóa chúng; template export ra thường hard-code giá trị và có thể không deploy lại được ngay; Bicep tự suy ra `dependsOn` khi tham chiếu symbol, ARM phải khai báo tay; `uniqueString(resourceGroup().id)` cho tên ổn định theo resource group; decompile là best effort.

Chi phí: vài cent.

Ngày 11 — Tạo VM, đổi size, quản lý disk, encryption at host

Bám bullet: tạo virtual machine; cấu hình encryption at host; quản lý VM size; quản lý VM disk.

Portal/CLI: đăng ký feature EncryptionAtHost, tạo VM Ubuntu B1s có auto-shutdown, gắn data disk, mở rộng disk, chụp snapshot.

```

az feature register --namespace Microsoft.Compute --name EncryptionAtHost
az vm create -g rg-az104-d11 -n vm-d11 --image Ubuntu2204 --size Standard_B1s
--admin-username azureuser --generate-ssh-keys
az vm auto-shutdown -g rg-az104-d11 -n vm-d11 --time 2300
az vm list-vm-resize-options -g rg-az104-d11 -n vm-d11 -o table
az vm disk attach -g rg-az104-d11 --vm-name vm-d11 --name disk-data1 --new --
size-gb 32 --sku StandardSSD_LRS
az snapshot create -g rg-az104-d11 -n snap-os --source $(az vm show -g rg-
az104-d11 -n vm-d11 --query storageProfile.osDisk.managedDisk.id -o tsv)

```

Lab C# (Day11): dùng VM, đổi size + bật encryption at host trong một lần patch, gắn disk mới, khởi động lại.

```

var rg = await Az.RgAsync("d11");
var vm = (await rg.GetVirtualMachines().GetAsync("vm-d11")).Value;

await foreach (var size in vm.GetAvailableSizesAsync())
    if (size.Name.StartsWith("Standard_B")) Console.WriteLine($"{size.Name}:
{size.NumberOfCores} vCPU, {size.MemoryInMB} MB");

await vm.DeallocateAsync(WaitUntil.Completed); // encryption at host và đổi
disk SKU cần deallocate
await vm.UpdateAsync(WaitUntil.Completed, new VirtualMachinePatch
{
    HardwareProfile = new VirtualMachineHardwareProfile { VmSize =
VirtualMachineSizeType.StandardB2S },
    SecurityProfile = new SecurityProfile { EncryptionAtHost = true }
});

var disk = (await
rg.GetManagedDisks().CreateOrUpdateAsync(WaitUntil.Completed, "disk-data2",
    new ManagedDiskData(AzureLocation.EastUS)
    {
        Sku = new DiskSku { Name = DiskStorageAccountType.StandardSsdLrs },
        DiskSizeGB = 16,
        CreationData = new DiskCreationData(DiskCreateOption.Empty)
    }
)).Value;
vm = (await vm.GetAsync()).Value;
vm.Data.StorageProfile.DataDisks.Add(new VirtualMachineDataDisk(2,
DiskCreateOptionType.Attach) { ManagedDisk = new VirtualMachineManagedDisk {
Id = disk.Id } });
vm = (await rg.GetVirtualMachines().CreateOrUpdateAsync(WaitUntil.Completed,

```

```
vm.Data.Name, vm.Data)).Value;
await vm.PowerOnAsync(WaitUntil.Completed);

foreach (var d in vm.Data.StorageProfile.DataDisks) Console.WriteLine($"LUN
{d.Lun}: {d.Name} {d.DiskSizeGB} GB");
```

Đọc template (ARM JSON): phần tài nguyên VM, đã lược bớt.

```
{
  "type": "Microsoft.Compute/virtualMachines",
  "apiVersion": "2024-03-01",
  "name": "[parameters('vmName')]",
  "location": "[parameters('location')]",
  "zones": [ "1" ],
  "dependsOn": [ "[resourceId('Microsoft.Network/networkInterfaces',
concat(parameters('vmName'), '-nic'))]" ],
  "properties": {
    "hardwareProfile": { "vmSize": "[parameters('vmSize')]" },
    "securityProfile": { "encryptionAtHost": false },
    "storageProfile": {
      "imageReference": { "publisher": "Canonical", "offer": "0001-com-
ubuntu-server-jammy", "sku": "22_04-lts-gen2", "version": "latest" },
      "osDisk": { "createOption": "FromImage", "managedDisk": {
"storageAccountType": "Standard_LRS" } },
      "dataDisks": [
        { "lun": 0, "createOption": "Empty", "diskSizeGB": 64, "managedDisk":
{ "storageAccountType": "StandardSSD_LRS" } }
      ]
    },
    "networkProfile": {
      "networkInterfaces": [ { "id": "
[resourceId('Microsoft.Network/networkInterfaces',
concat(parameters('vmName'), '-nic'))]" } ]
    }
  }
}
```

1. Cần đặt VM vào zone 2 và đồng thời vào availability set `avset1` . Làm được không?
2. Thêm một data disk 128 GB Premium SSD thì thêm gì vào template?
3. Muốn bật encryption at host, sửa dòng nào và cần điều kiện gì?
4. `dependsOn` ở đây đảm bảo điều gì?

Đáp án: (1) Đổi `zones` thành `[ "2" ]` được, nhưng không thể vừa có zone vừa thuộc availability set; phải chọn một. (2) Thêm phần tử `{ "lun": 1, "createOption": "Empty", "diskSizeGB": 128, "managedDisk": { "storageAccountType": "Premium_LRS" } }`; `lun` phải khác các disk đã có, và VM size phải hỗ trợ Premium storage. (3) `"encryptionAtHost": true`; subscription phải đã đăng ký feature `EncryptionAtHost`, VM đang chạy thì phải deallocate trước. (4) NIC được tạo xong trước khi tạo VM.

Bây đề thi: đổi size có thể cần deallocate nếu size mới không có trên cụm phần cứng hiện tại; disk chỉ tăng được, không thu nhỏ; temp disk mất dữ liệu khi deallocate; ba lớp mã hoá để nhầm: SSE (mặc định, at rest trên storage), encryption at host (mã hoá cả temp disk và cache trên host), Azure Disk Encryption (BitLocker/DM-Crypt trong OS); đổi disk Standard sang Premium cần VM size hỗ trợ Premium storage.

Chi phí: ~\$0.5.

Ngày 12 — Availability zone/set, di chuyển VM, VM Scale Sets

Bám bullet: deploy VM vào availability zone và availability set; di chuyển VM sang resource group, subscription hoặc region khác; deploy và cấu hình VM Scale Sets.

Portal/CLI: tạo availability set (2 fault domain, 5 update domain) với 2 VM B1s; tạo VMSS Uniform 2 instance trải qua zone 1 và 2, autoscale theo CPU; xem Azure Resource Mover cho trường hợp đổi region.

```
az vmss create -g rg-az104-d12 -n vmss-d12 --image Ubuntu2204 --vm-sku
Standard_B1s --instance-count 2 \
  --zones 1 2 --orchestration-mode Uniform --admin-username azureuser --
generate-ssh-keys
az monitor autoscale create -g rg-az104-d12 --resource vmss-d12 --resource-
type Microsoft.Compute/virtualMachineScaleSets \
  --name as-d12 --min-count 1 --max-count 3 --count 2
az monitor autoscale rule create -g rg-az104-d12 --autoscale-name as-d12 --
condition "Percentage CPU > 70 avg 5m" --scale out 1
```

Lab C# (Day12): gom VM cùng mọi tài nguyên phụ thuộc rồi validate + move sang resource group khác; scale VMSS và liệt kê instance theo zone.

```
var src = await Az.RgAsync("d12");
var dst = await Az.RgAsync("d12-moved");
var vm = (await src.GetVirtualMachines().GetAsync("vm-avset-1")).Value;
```

```
// VM không đi một mình: phải kèm OS disk, NIC, public IP
var ids = new List<ResourceIdentifier> { vm.Id,
vm.Data.StorageProfile.OSDisk.ManagedDisk.Id };
foreach (var nicRef in vm.Data.NetworkProfile.NetworkInterfaces)
{
    var nic = (await
Az.Arm.GetNetworkInterfaceResource(nicRef.Id).GetAsync()).Value;
    ids.Add(nic.Id);
    foreach (var ip in nic.Data.IPConfigurations)
        if (ip.PublicIPAddress?.Id is { } pip) ids.Add(pip);
}
var move = new ResourcesMoveContent { TargetResourceGroupId = dst.Id };
foreach (var id in ids) move.Resources.Add(id);
try { await src.ValidateMoveResourcesAsync(WaitUntil.Completed, move);
Console.WriteLine("Move hợp lệ"); }
catch (RequestFailedException ex) { Console.WriteLine($"Không move được:
{ex.Message}"); } // ví dụ: VM trong availability set phải đi cùng set

// VMSS
var vmss = (await src.GetVirtualMachineScaleSets().GetAsync("vmss-
d12")).Value;
await vmss.UpdateAsync(WaitUntil.Completed, new VirtualMachineScaleSetPatch {
Sku = new ComputeSku { Name = vmss.Data.Sku.Name, Capacity = 3 } });
await foreach (var inst in vmss.GetVirtualMachineScaleSetVms().GetAllAsync())
    Console.WriteLine($"{inst.Data.Name} zone={string.Join(',',
inst.Data.Zones)} state={inst.Data.ProvisioningState}");
```

Bắt đề thi: SLA: 2+ VM qua availability zone 99,99%, qua availability set 99,95%, VM đơn dùng Premium SSD 99,9%; VM chỉ vào availability set lúc tạo, không thêm sau được; một VM không thể vừa ở set vừa ở zone; move giữa resource group/subscription không đổi region, đổi region dùng Resource Mover hoặc Site Recovery; move sang subscription khác phải cùng tenant; VMSS Flexible cho phép trộn size và quản lý từng VM như VM thường.

Chi phí: ~\$1.

Ngày 13 — Container: ACR, ACI, Container Apps, sizing và scaling

Bám bullet: tạo và quản lý Azure Container Registry; provision container bằng Azure Container Instances; provision container bằng Azure Container Apps; quản lý sizing và scaling cho ACI và Container Apps.

Portal/CLI: tạo ACR Basic, import image mẫu, chạy bằng ACI (1 vCPU, 1.5 GB, restart policy OnFailure, DNS label), rồi chạy cùng image bằng Container Apps với HTTP scale rule.

```

az acr create -g rg-az104-d13 -n acr104$RANDOM --sku Basic
az acr import -n <acr> --source mcr.microsoft.com/azuredocs/aci-helloworld:latest --image hello:v1
az container create -g rg-az104-d13 -n aci-hello --image mcr.microsoft.com/azuredocs/aci-helloworld \
    --cpu 1 --memory 1.5 --restart-policy OnFailure --ports 80 --dns-name-label aci104$RANDOM --os-type Linux
az containerapp up -g rg-az104-d13 -n aca-hello --image mcr.microsoft.com/azuredocs/containerapps-helloworld:latest \
    --ingress external --target-port 80

```

Lab C# (Day13): tạo ACI bằng SDK, sửa scale rule của Container App, rồi bắn tải để thấy replica tăng.

```

var rg = await Az.RgAsync("d13");
var cg = new ContainerGroupData(AzureLocation.EastUS,
    [ new ContainerInstanceContainer("hello",
        "mcr.microsoft.com/azuredocs/aci-helloworld",
        new ContainerResourceRequirements(new
            ContainerResourceRequestsContent(memoryInGB: 1.5, cpu: 1)))
        { Ports = { new ContainerPort(80) } } ],
        ContainerInstanceOperatingSystemType.Linux)
{
    RestartPolicy = ContainerGroupRestartPolicy.OnFailure,
    IPAddress = new ContainerGroupIPAddress([ new ContainerGroupPort(80) ],
        ContainerGroupIPAddressType.Public) { DnsNameLabel =
        $"aci104sdk{Random.Shared.Next(9999)}" }
};
var group = (await
    rg.GetContainerGroups().CreateOrUpdateAsync(WaitUntil.Completed, "aci-sdk",
        cg)).Value;
Console.WriteLine($"http://{group.Data.IPAddress.Fqdn}");

// Container Apps: 0..5 replica, scale theo 20 request đồng thời
var app = (await rg.GetContainerApps().GetAsync("aca-hello")).Value;
var data = app.Data;
data.Template.Scale = new ContainerAppScale
{
    MinReplicas = 0, MaxReplicas = 5,
    Rules = { new ContainerAppScaleRule { Name = "http-load", Http = new
        ContainerAppHttpScaleRule { Metadata = { ["concurrentRequests"] = "20" } } } }
}

```

```
};
await app.UpdateAsync(WaitUntil.Completed, data);

// Bắn tải 3.000 request, rồi chạy: az containerapp replica list -g ... -n
aca-hello
using var http = new HttpClient();
await Parallel.ForEachAsync(Enumerable.Range(0, 3000), new ParallelOptions {
    MaxDegreeOfParallelism = 100 },
    async (_, ct) => await
    http.GetAsync($"https://{data.Configuration.Ingress.Fqdn}", ct));
```

Bây đề thi: ACI không có autoscale, muốn đổi CPU/RAM phải tạo lại container group; các container trong một group chia sẻ vòng đời, mạng và volume; restart policy Always/OnFailure/Never; Container Apps scale từ 0 theo HTTP, TCP hoặc rule KEDA; ACR Basic/Standard/Premium khác nhau ở dung lượng, geo-replication và private link (chỉ Premium).

Chi phí: ~\$0.5.

Ngày 14 — App Service: plan, scaling, tạo app, backup, deployment slot

Bám bullet: provision App Service plan; cấu hình scaling cho plan; tạo App Service; cấu hình backup; cấu hình deployment slot.

Portal/CLI: plan Linux S1, web app .NET 8, autoscale theo CPU, backup thủ công vào storage container, slot **staging** có app setting **Banner** đánh dấu là slot setting, swap rồi swap ngược.

```
az appservice plan create -g rg-az104-d14 -n plan-d14 --is-linux --sku S1
az webapp create -g rg-az104-d14 -p plan-d14 -n app104$RANDOM --runtime
"DOTNETCORE:8.0"
az webapp deployment slot create -g rg-az104-d14 -n <app> --slot staging
az webapp config appsettings set -g rg-az104-d14 -n <app> --slot staging --
slot-settings Banner=STAGING
az monitor autoscale create -g rg-az104-d14 --resource plan-d14 --resource-
type Microsoft.Web/serverfarms \
    --name as-plan --min-count 1 --max-count 3 --count 1
az webapp config backup create -g rg-az104-d14 --webapp-name <app> --
container-url "<container-SAS-url>" --backup-name manual1
```

Lab C# (Day14): web app nhỏ hiển thị **Banner**, tạo slot và swap bằng SDK, scale out plan.

```
// Web app (deploy bằng: az webapp up hoặc zip deploy vào cả 2 slot)
app.MapGet("/", (IConfiguration c) => $"Banner = {c["Banner"]} ??
"PRODUCTION"}), host = {Environment.MachineName}");

// AzAdminKit Day14
var rg = await Az.RgAsync("d14");
var plan = (await rg.GetAppServicePlans().GetAsync("plan-d14")).Value;
plan.Data.Sku.Capacity = 2; //
scale out thủ công
await rg.GetAppServicePlans().CreateOrUpdateAsync(WaitUntil.Completed,
plan.Data.Name, plan.Data);

var site = (await rg.GetWebSites().GetAsync("<app>")).Value;
await foreach (var s in site.GetWebSiteSlots().GetAllAsync())
Console.WriteLine($"slot: {s.Data.Name}");
await site.SwapSlotWithProductionAsync(WaitUntil.Completed, new
CsmSlotEntity("staging", preserveVnet: true));
```

Sau swap, mở URL production: code của staging đã lên, nhưng **Banner** vẫn là PRODUCTION vì là slot setting.

Bãi đề thi: scale up (đổi tier) khác scale out (thêm instance); autoscale và slot cần Standard trở lên (Standard 5 slot, Premium 20 slot); autoscale đặt trên plan, áp cho mọi app trong plan; mọi slot dùng chung tài nguyên của plan; swap with preview cho phép kiểm tra trước khi hoàn tất; backup có giới hạn dung lượng và cần chọn tier hỗ trợ.

Chi phí: ~\$0.5.

Ngày 15 — App Service: TLS, custom domain, networking

Bám bullet: cấu hình certificate và TLS cho App Service; map custom DNS name có sẵn vào App Service; cấu hình networking cho App Service.

Portal: nếu có tên miền (có thể mua tên miền rẻ), thêm custom domain với bản ghi CNAME + TXT **asuid**, tạo App Service Managed Certificate, bind SNI SSL; bật HTTPS only, TLS tối thiểu 1.2; thêm access restriction chỉ cho IP của bạn; tạo VNet và bật VNet integration. Không có tên miền thì vẫn làm đủ phần TLS và networking, còn phần DNS làm ở ngày 18.

Lab C# (Day15): khoá app bằng TLS 1.2 + HTTPS only + access restriction, và in giá trị cần đặt cho bản ghi xác minh tên miền.

```

var site = (await rg.GetWebSites().GetAsync("<app>")).Value;
Console.WriteLine($"TXT asuid.<sub> =
{site.Data.CustomDomainVerificationId}");
Console.WriteLine($"CNAME <sub> -> {site.Data.DefaultHostName}");

await site.UpdateAsync(new SitePatchInfo { IsHttpsOnly = true });

var cfg = (await site.GetWebSiteConfig().GetAsync()).Value;
cfg.Data.MinTlsVersion = AppServiceSupportedTlsVersion.Tls1_2;
cfg.Data.IPSecurityRestrictions.Add(new AppServiceIPSecurityRestriction
{
    Name = "allow-me", IPAddressOrCidr = $"{myPublicIp}/32", Action =
    "Allow", Priority = 100
});
await cfg.CreateOrUpdateAsync(WaitUntil.Completed, cfg.Data);

// Kiểm chứng: gọi bằng HTTP thường phải bị chuyển hướng 301 sang HTTPS
using var http = new HttpClient(new HttpClientHandler { AllowAutoRedirect =
false });
Console.WriteLine((await
http.GetAsync($"http://{site.Data.DefaultHostName}")).StatusCode);

```

Bắt đề thi: subdomain dùng CNAME, tên miền gốc (apex) dùng A record kèm TXT `asuid`; App Service Managed Certificate miễn phí nhưng không hỗ trợ wildcard; custom domain và TLS cần tier Basic trở lên; VNet integration chỉ cho traffic đi ra, muốn truy cập vào app qua IP riêng thì dùng private endpoint; access restriction xử lý theo priority, số nhỏ trước.

Chi phí: ~\$0.5 (chưa tính tên miền).

Ngày 16–19: Implement and manage virtual networking (15–20%)

Bốn ngày cho 13 gạch đầu dòng, dựng dần một mô hình hub-spoke trong `rg-az104-net` giữ từ ngày 16 đến 19 (VNet miễn phí; chỉ VM, public IP, private endpoint và load balancer tính phí). Dùng VM qua đêm bằng auto-shutdown.

```

vnet-hub    10.0.0.0/16    snet-shared 10.0.1.0/24 | AzureBastionSubnet
10.0.254.0/26
vnet-spoke  10.1.0.0/16    snet-app    10.1.1.0/24 | snet-db 10.1.2.0/24 |
snet-pe 10.1.3.0/24

```

Ngày 16 — VNet, subnet, peering, public IP, user-defined route

Bám bullet: tạo và cấu hình virtual network và subnet; tạo và cấu hình VNet peering; cấu hình public IP address; cấu hình user-defined route.

Portal/CLI: dựng hai VNet như sơ đồ, peering hai chiều, tạo public IP Standard tĩnh zone-redundant, route table đẩy 0.0.0.0/0 của snet-app về một NVA giả định 10.0.1.4.

```
az network vnet peering create -g rg-az104-net -n spoke-to-hub --vnet-name
vnet-spoke --remote-vnet vnet-hub \
  --allow-vnet-access --allow-forwarded-traffic
az network public-ip create -g rg-az104-net -n pip-std --sku Standard --
allocation-method Static --zone 1 2 3
az network route-table route create -g rg-az104-net --route-table-name rt-
spoke -n default-to-nva \
  --address-prefix 0.0.0.0/0 --next-hop-type VirtualAppliance --next-hop-ip-
address 10.0.1.4
```

Lab C# (Day16): dựng toàn bộ hub-spoke bằng SDK và tự tính số IP dùng được của mỗi subnet.

```
var rg = await Az.RgAsync("net");
async Task<VirtualNetworkResource> VnetAsync(string name, string space,
params (string Name, string Prefix)[] subnets)
{
    var data = new VirtualNetworkData { Location = AzureLocation.EastUS,
AddressPrefixes = { space } };
    foreach (var s in subnets) data.Subnets.Add(new SubnetData { Name =
s.Name, AddressPrefix = s.Prefix });
    return (await
rg.GetVirtualNetworks().CreateOrUpdateAsync(WaitUntil.Completed, name,
data)).Value;
}
var hub = await VnetAsync("vnet-hub", "10.0.0.0/16", ("snet-shared",
"10.0.1.0/24"), ("AzureBastionSubnet", "10.0.254.0/26"));
var spoke = await VnetAsync("vnet-spoke", "10.1.0.0/16", ("snet-app",
"10.1.1.0/24"), ("snet-db", "10.1.2.0/24"), ("snet-pe", "10.1.3.0/24"));

// Peering phải tạo ở CẢ HAI phía
foreach (var (a, b) in new[] { (hub, spoke), (spoke, hub) })
    await
a.GetVirtualNetworkPeerings().CreateOrUpdateAsync(WaitUntil.Completed, $"
{a.Data.Name}-to-{b.Data.Name}",
```

```

        new VirtualNetworkPeeringData { RemoteVirtualNetworkId = b.Id,
        AllowVirtualNetworkAccess = true, AllowForwardedTraffic = true });

// UDR gắn vào snet-app
var rt = (await rg.GetRouteTables().CreateOrUpdateAsync(WaitUntil.Completed,
"rt-spoke", new RouteTableData
{
    Location = AzureLocation.EastUS, DisableBgpRoutePropagation = true,
    Routes = { new RouteData { Name = "default-to-nva", AddressPrefix =
"0.0.0.0/0", NextHopType = RouteNextHopType.VirtualAppliance,
NextHopIPAddress = "10.0.1.4" } }
})).Value;
var app = (await spoke.GetSubnets().GetAsync("snet-app")).Value;
app.Data.RouteTable = new RouteTableData { Id = rt.Id };
await spoke.GetSubnets().CreateOrUpdateAsync(WaitUntil.Completed, "snet-app",
app.Data);

// Azure giữ 5 IP mỗi subnet
await foreach (var s in spoke.GetSubnets().GetAllAsync())
{
    var net = System.Net.IPNetwork.Parse(s.Data.AddressPrefix);
    Console.WriteLine($"{s.Data.Name,-12} {s.Data.AddressPrefix,-14} usable=
{Math.Pow(2, 32 - net.PrefixLength) - 5}");
}

```

Bẫy đề thi: peering không bắc cầu (A-B, B-C không suy ra A-C); address space hai VNet không được chồng lấn; route chọn theo prefix dài nhất, cùng prefix thì UDR > BGP > system route; next hop type gồm VirtualAppliance, VirtualNetworkGateway, Internet, None, VnetLocal; public IP SKU Basic đã ngừng, Standard luôn là Static; gateway transit cho phép spoke dùng VPN gateway của hub.

Chi phí: vài cent (public IP).

Ngày 17 — NSG, ASG, effective security rules, Azure Bastion

Bám bullet: tạo và cấu hình network security group và application security group; đánh giá effective security rules trong NSG; triển khai Azure Bastion.

Portal/CLI: 2 VM B1s trong `snet-app` (gắn ASG `asg-web`) và `snet-db` (gắn ASG `asg-db`); NSG cho phép 80 từ Internet vào `asg-web`, 1433 từ `asg-web` vào `asg-db`; triển khai Bastion SKU Developer (miễn phí) rồi thử Basic để so sánh; SSH qua Bastion, không gán public IP cho VM.

Lab C# (Day17): tạo ASG + NSG có rule tham chiếu ASG, rồi in effective rules thật sự áp lên NIC.

```
var asgWeb = (await
rg.GetApplicationSecurityGroups().CreateOrUpdateAsync(WaitUntil.Completed,
"asg-web", new ApplicationSecurityGroupData { Location = AzureLocation.EastUS
})).Value;
var asgDb = (await
rg.GetApplicationSecurityGroups().CreateOrUpdateAsync(WaitUntil.Completed,
"asg-db", new ApplicationSecurityGroupData { Location = AzureLocation.EastUS
})).Value;

var nsgData = new NetworkSecurityGroupData { Location = AzureLocation.EastUS
};
nsgData.SecurityRules.Add(new SecurityRuleData
{
    Name = "allow-http-web", Priority = 100, Direction =
SecurityRuleDirection.Inbound, Access = SecurityRuleAccess.Allow,
    Protocol = SecurityRuleProtocol.Tcp, SourceAddressPrefix = "Internet",
    SourcePortRange = "*", DestinationPortRange = "80",
    DestinationApplicationSecurityGroups = { asgWeb.Data }
});
nsgData.SecurityRules.Add(new SecurityRuleData
{
    Name = "allow-sql-web-to-db", Priority = 110, Direction =
SecurityRuleDirection.Inbound, Access = SecurityRuleAccess.Allow,
    Protocol = SecurityRuleProtocol.Tcp, SourcePortRange = "*",
    DestinationPortRange = "1433",
    SourceApplicationSecurityGroups = { asgWeb.Data },
    DestinationApplicationSecurityGroups = { asgDb.Data }
});
var nsg = (await
rg.GetNetworkSecurityGroups().CreateOrUpdateAsync(WaitUntil.Completed, "nsg-
spoke", nsgData)).Value;

// Effective rules = default rules + NSG subnet + NSG NIC, đã gộp (VM phải
đang chạy)
var nic = (await rg.GetNetworkInterfaces().GetAsync("vm-webVMNic")).Value;
var eff = await
nic.GetEffectiveNetworkSecurityGroupsAsync(WaitUntil.Completed);
foreach (var g in eff.Value.Value)
```

```

    foreach (var r in g.EffectiveSecurityRules.OrderBy(r =>
r.Direction.ToString()).ThenBy(r => r.Priority))
        Console.WriteLine($"{r.Direction,-8} {r.Priority,5} {r.Access,-5}
        {r.Name}");

```

Đọc template (Bicep): VNet ba subnet kèm NSG.

```

param location string = resourceGroup().location
var subnetNames = ['web', 'app', 'db']

resource nsgWeb 'Microsoft.Network/networkSecurityGroups@2023-11-01' = {
  name: 'nsg-web'
  location: location
  properties: {
    securityRules: [
      {
        name: 'allow-https'
        properties: { priority: 200, direction: 'Inbound', access: 'Allow',
protocol: 'Tcp'
        sourceAddressPrefix: 'Internet', sourcePortRange: '*',
destinationAddressPrefix: '*', destinationPortRange: '443' }
      }
      {
        name: 'deny-all-in'
        properties: { priority: 150, direction: 'Inbound', access: 'Deny',
protocol: '*'
        sourceAddressPrefix: '*', sourcePortRange: '*',
destinationAddressPrefix: '*', destinationPortRange: '*' }
      }
    ]
  }
}

resource vnet 'Microsoft.Network/virtualNetworks@2023-11-01' = {
  name: 'vnet-app'
  location: location
  properties: {
    addressSpace: { addressPrefixes: ['10.20.0.0/16'] }
    subnets: [for (s, i) in subnetNames: {
      name: 'snet-${s}'
      properties: {
        addressPrefix: '10.20.${i}.0/24'
        networkSecurityGroup: s == 'web' ? { id: nsgWeb.id } : null

```

```
}
  }
}
}
```

1. Template tạo bao nhiêu subnet, tên và dải địa chỉ của từng subnet là gì?
2. Người dùng Internet không vào được HTTPS của VM trong `snet-web` . Vì sao?
3. Subnet nào có NSG gắn vào?
4. Có cần thêm `dependsOn` để NSG được tạo trước VNet không?

Đáp án: (1) 3 subnet: `snet-web` 10.20.0.0/24, `snet-app` 10.20.1.0/24, `snet-db` 10.20.2.0/24. (2) Rule `deny-all-in` priority 150 được xét trước `allow-https` priority 200; đổi allow xuống priority nhỏ hơn 150, hoặc bỏ hẳn rule deny vì rule mặc định `DenyAllInBound` đã chặn phần còn lại. (3) Chỉ `snet-web` . (4) Không, Bicep tự suy ra phụ thuộc vì có tham chiếu `nsgWeb.id` .

Bẫy đề thi: inbound đi qua NSG của subnet trước rồi NSG của NIC, outbound theo chiều ngược lại, và phải được cả hai cho phép; priority 100–4096, số nhỏ thắng; ba rule mặc định inbound (`AllowVnetInBound`, `AllowAzureLoadBalancerInBound`, `DenyAllInBound`) không xoá được, chỉ ghi đè bằng priority nhỏ hơn; ASG chỉ dùng cho NIC trong cùng VNet; Bastion cần subnet tên đúng `AzureBastionSubnet` , tối thiểu /26; kết nối bằng native client cần SKU Standard trở lên.

Chi phí: ~\$0.5.

Ngày 18 — Service endpoint, private endpoint, Azure DNS

Bám bullet: cấu hình service endpoint cho PaaS; cấu hình private endpoint cho PaaS; cấu hình Azure DNS.

Portal/CLI: storage A dùng service endpoint `Microsoft.Storage` trên `snet-app` kèm network rule cho subnet đó; storage B dùng private endpoint trong `snet-pe` với private DNS zone `privatelink.blob.core.windows.net` link vào cả hai VNet; tạo public DNS zone (nếu có tên miền thì cập nhật NS ở nhà đăng ký) và private zone `corp.lab` bật auto-registration cho `vnet-spoke` .

```
az network vnet subnet update -g rg-az104-net --vnet-name vnet-spoke -n snet-app --service-endpoints Microsoft.Storage
az storage account network-rule add -g rg-az104-net --account-name <stA> --vnet-name vnet-spoke --subnet snet-app
az network private-endpoint create -g rg-az104-net -n pe-stB --vnet-name vnet-spoke --subnet snet-pe \
    --private-connection-resource-id <stB-id> --group-id blob --connection-name pe-stB-conn
az network private-dns zone create -g rg-az104-net -n privatelink.blob.core.windows.net
az network private-dns link vnet create -g rg-az104-net -z privatelink.blob.core.windows.net -n link-spoke -v vnet-spoke -e false
az network private-endpoint dns-zone-group create -g rg-az104-net --endpoint-name pe-stB -n default \
    --private-dns-zone privatelink.blob.core.windows.net --zone-name blob
az network private-dns zone create -g rg-az104-net -n corp.lab
az network private-dns link vnet create -g rg-az104-net -z corp.lab -n reg-spoke -v vnet-spoke -e true
```

Lab C# (Day18): tạo bản ghi trong public DNS zone, rồi chạy `nslookup` bên trong VM để so sánh kết quả phân giải của storage A (IP public) và storage B (IP private).

```

var zone = (await rg.GetDnsZones().CreateOrUpdateAsync(WaitUntil.Completed,
"az104-lab.example.com", new DnsZoneData("global"))).Value;
Console.WriteLine($"Name servers: {string.Join(", ",
zone.Data.NameServers)}");
await zone.GetDnsARecords().CreateOrUpdateAsync(WaitUntil.Completed, "www",
    new DnsARecordData { TtlInSeconds = 300, DnsARecords = { new
DnsARecordInfo { IPv4Address = IPAddress.Parse("20.1.2.3") } } });
await zone.GetDnsCnameRecords().CreateOrUpdateAsync(WaitUntil.Completed,
"app",
    new DnsCnameRecordData { TtlInSeconds = 300, Cname = "
<app>.azurewebsites.net" });

// nslookup từ trong VM
var vm = (await rg.GetVirtualMachines().GetAsync("vm-web")).Value;
var run = await vm.RunCommandAsync(WaitUntil.Completed, new
RunCommandInput("RunShellScript")
{
    Script = { $"nslookup {stA}.blob.core.windows.net", $"nslookup
{stB}.blob.core.windows.net", "nslookup vm-db.corp.lab" }
});
foreach (var s in run.Value.Value) Console.WriteLine(s.Message);

```

Bây đề thi: service endpoint vẫn dùng IP public của dịch vụ, chỉ tối ưu đường đi và cho phép lọc theo subnet, không dùng được từ on-prem; private endpoint cấp IP private trong VNet, truy cập được qua peering/VPN, và gần như luôn cần private DNS zone; một VNet chỉ link registration (auto-registration) với một private zone; public zone chỉ hoạt động khi nhà đăng ký trở NS về Azure DNS; alias record trở thẳng vào tài nguyên Azure và tự cập nhật khi IP đổi.

Chi phí: ~\$0.5.

Ngày 19 — Load balancer và troubleshooting mạng

Bám bullet: cấu hình internal hoặc public load balancer; troubleshoot load balancing; troubleshoot kết nối mạng.

Portal/CLI: Standard public LB, backend pool 2 VM chạy nginx (cài qua custom data), health probe HTTP / , rule cổng 80; một internal LB Standard cho snet-db . Cố tình tắt nginx trên một VM, và thêm một rule NSG deny cổng 80 priority 90, rồi chẩn đoán bằng Network Watcher.

Lab C# (Day19): ba công cụ Network Watcher bằng SDK và đọc metric sức khỏe backend của LB.

```

var sub = await Az.Arm.GetDefaultSubscriptionAsync();
var nwRg = (await sub.GetResourceGroupAsync("NetworkWatcherRG")).Value;
var nw = (await
nwRg.GetNetworkWatchers().GetAsync("NetworkWatcher_eastus")).Value;
var vm = (await rg.GetVirtualMachines().GetAsync("vm-web")).Value;

// 1) IP flow verify: rule nào chặn gói tin vào cổng 80?
var flow = await nw.VerifyIPFlowAsync(WaitUntil.Completed, new
VerificationIPFlowContent(
    vm.Id, NetworkTrafficDirection.Inbound, IPFlowProtocol.Tcp, localPort:
"80", remotePort: "60000",
    localIPAddress: "10.1.1.4", remoteIPAddress: "203.0.113.10"));
Console.WriteLine($"IP flow: {flow.Value.Access} by {flow.Value.RuleName}");

// 2) Next hop: traffic ra Internet đi đâu (sau UDR ngày 16 sẽ là
VirtualAppliance)
var hop = await nw.GetNextHopAsync(WaitUntil.Completed, new
NextHopContent(vm.Id, "10.1.1.4", "8.8.8.8"));
Console.WriteLine($"Next hop: {hop.Value.NextHopType}
{hop.Value.NextHopIPAddress} route={hop.Value.RouteTableId}");

// 3) Connection troubleshoot đầu-cuối
var conn = await nw.CheckConnectivityAsync(WaitUntil.Completed,
    new ConnectivityContent(new ConnectivitySource(vm.Id), new
ConnectivityDestination { Address = "www.microsoft.com", Port = 443 }));
Console.WriteLine($"Connectivity: {conn.Value.ConnectionStatus}, hops=
{conn.Value.Hops.Count}");

// 4) Sức khỏe backend của LB (NuGet: Azure.Monitor.Query)
var metrics = new MetricsQueryClient(Az.Cred);
var lbId = (await rg.GetLoadBalancers().GetAsync("lb-
web")).Value.Id.ToString();
var res = await metrics.QueryResourceAsync(lbId, ["DipAvailability"],
    new MetricsQueryOptions { TimeRange = new
QueryTimeRange(TimeSpan.FromMinutes(30)), Granularity =
TimeSpan.FromMinutes(5) });
foreach (var ts in res.Value.Metrics[0].TimeSeries)
    foreach (var p in ts.Values) Console.WriteLine($"{p.TimeStamp:t} health
probe = {p.Average}%");

```

Đọc template (ARM JSON): load balancer, đã lược bớt.

```
{
  "type": "Microsoft.Network/loadBalancers",
  "apiVersion": "2023-11-01",
  "name": "lb-web",
  "location": "[parameters('location')]",
  "sku": { "name": "Standard" },
  "properties": {
    "frontendIPConfigurations": [
      { "name": "fe", "properties": { "publicIPAddress": { "id": "[resourceId('Microsoft.Network/publicIPAddresses', 'pip-lb')]" } } },
    ],
    "backendAddressPools": [ { "name": "be-web" } ],
    "probes": [
      { "name": "hp-http", "properties": { "protocol": "Http", "port": 8080, "requestPath": "/healthz", "intervalInSeconds": 5, "numberOfProbes": 2 } } ],
    "loadBalancingRules": [ {
      "name": "rule-http",
      "properties": {
        "frontendIPConfiguration": { "id": "[resourceId('Microsoft.Network/loadBalancers/frontendIPConfigurations', 'lb-web', 'fe')]" },
        "backendAddressPool": { "id": "[resourceId('Microsoft.Network/loadBalancers/backendAddressPools', 'lb-web', 'be-web')]" },
        "probe": { "id": "[resourceId('Microsoft.Network/loadBalancers/probes', 'lb-web', 'hp-http')]" },
        "protocol": "Tcp", "frontendPort": 80, "backendPort": 80,
        "loadDistribution": "SourceIP", "idleTimeoutInMinutes": 4
      }
    } ]
  }
}
```

1. Đây là load balancer public hay internal? Muốn đổi sang internal thì sửa phần nào?
2. Các VM backend chạy nginx chỉ ở cổng 80, và LB báo mọi backend đều unhealthy. Vì sao?
3. "loadDistribution": "SourceIP" có tác dụng gì?
4. Public IP pip-lb phải dùng SKU nào?

Đáp án: (1) Public, vì frontend dùng `publicIPAddress` ; đổi sang internal bằng cách thay bằng `subnet` (kèm `privateIPAddress` hoặc cấp động). (2) Probe kiểm tra cổng 8080 đường dẫn `/healthz` , trong khi nginx chỉ nghe cổng 80; sửa probe về `port: 80` và một đường dẫn có thật. (3) Session persistence theo IP client: cùng một IP nguồn luôn vào cùng một backend. (4) Standard, phải cùng SKU với load balancer.

Bây đề thi: Standard LB đóng mặc định, cần NSG cho phép traffic vào backend; health probe đến từ IP `168.63.129.16` (service tag AzureLoadBalancer), chặn IP này thì mọi backend bị coi là chết; LB là tầng 4, cần định tuyến theo URL thì dùng Application Gateway; backend của một LB phải nằm trong cùng VNet; HA ports chỉ có ở internal Standard LB; IP flow verify trả lời "rule nào chặn", next hop trả lời "gói tin đi đâu", connection troubleshoot kiểm tra cả đường đi lẫn đích.

Chi phí: ~\$1. Xóa `rg-az104-net` cuối ngày.

Ngày 20–22: Monitor and maintain Azure resources (10–15%)

Ba ngày cho 13 gạch đầu dòng. Giữ `rg-az104-mon` (Log Analytics workspace, 1 VM B1s, vault) từ ngày 20 đến 22, vì backup cần chạy xong lần đầu và Site Recovery cần thời gian replicate ban đầu. Lab C# chủ yếu dùng `Azure.Monitor.Query` để đọc metric và log, đúng thứ admin làm hằng ngày.

Ngày 20 — Metric, diagnostic settings, KQL, alert

Bám bullet: diễn giải metric trong Azure Monitor; cấu hình log settings; query và phân tích log; thiết lập alert rule, action group và alert processing rule.

Portal/CLI: tạo workspace; diagnostic setting cho blob service của một storage account và cho Activity log, đẩy về workspace; metric alert CPU > 80% trên VM; log alert khi có thao tác xóa tài nguyên; action group gửi email; alert processing rule tắt thông báo trong khung giờ bảo trì tối thứ Bảy.

```

az monitor log-analytics workspace create -g rg-az104-mon -n law-az104
az monitor action-group create -g rg-az104-mon -n ag-admin --short-name admin
--action email me <email>
az monitor metrics alert create -g rg-az104-mon -n cpu-high --scopes <vmId> -
-condition "avg Percentage CPU > 80" \
  --window-size 5m --evaluation-frequency 1m --action ag-admin
az monitor alert-processing-rule create -g rg-az104-mon -n maint-sat --rule-
type RemoveAllActionGroups \
  --scopes /subscriptions/<subId>/resourceGroups/rg-az104-mon --schedule-
recurrence-type Weekly \
  --schedule-recurrence Saturday --schedule-start-datetime "2026-10-03
20:00:00" --schedule-end-datetime "2026-12-31 23:00:00"

```

Lab C# (Day20): bật diagnostic setting bằng SDK, đọc metric CPU, chạy KQL trên workspace.

```

var rg = await Az.RgAsync("mon");
var law = (await rg.GetOperationalInsightsWorkspaces().GetAsync("law-
az104")).Value;

// Diagnostic setting cho blob service
var blobScope = new ResourceIdentifier($"{storageId}/blobServices/default");
await
Az.Arm.GetDiagnosticSettings(blobScope).CreateOrUpdateAsync(WaitUntil.Comple
ted, "to-law", new DiagnosticSettingData
{
    WorkspaceId = law.Id,
    Logs = { new LogSettings(true) { CategoryGroup = "allLogs" } },
    Metrics = { new MetricSettings(true) { Category = "Transaction" } }
});

// Metric: CPU 1 giờ qua, Average và Maximum (thường lệch nhau rất nhiều, đề
hay hỏi chọn aggregation nào)
var metrics = new MetricsQueryClient(Az.Cred);
var m = await metrics.QueryResourceAsync(vmId, ["Percentage CPU"], new
MetricsQueryOptions
{
    TimeRange = new QueryTimeRange(TimeSpan.FromHours(1)), Granularity =
TimeSpan.FromMinutes(5),
    Aggregations = { MetricAggregationType.Average,
MetricAggregationType.Maximum }
});

```

```
foreach (var p in m.Value.Metrics[0].TimeSeries[0].Values)
    Console.WriteLine($"{p.TimeStamp:t} avg={p.Average:F1} max={p.Maximum:F1}");

// Logs: ai đã xóa gì trong 24 giờ qua
var logs = new LogsQueryClient(Az.Cred);
var r = await logs.QueryWorkspaceAsync(law.Data.CustomerId.ToString(), ""
    AzureActivity
    | where OperationNameValue endswith "/DELETE" and ActivityStatusValue ==
    "Success"
    | project TimeGenerated, Caller, ResourceGroup, Resource =
    tostring(split(_ResourceId, "/")[-1])
    | order by TimeGenerated desc
    "", new QueryTimeRange(TimeSpan.FromDays(1)));
foreach (var row in r.Value.Table.Rows) Console.WriteLine(string.Join(" | ",
    row));
```

Viết thêm bằng tay ba truy vấn trên portal: `StorageBlobLogs` đếm theo `OperationName` và `StatusText`; `Heartbeat` tìm VM mất tín hiệu quá 10 phút; `Perf` hoặc `InsightsMetrics` vẽ CPU bằng `render timechart`.

Bây đề thi: platform metric tự có và lưu 93 ngày, còn resource log chỉ có khi tạo diagnostic setting; Activity log lưu 90 ngày; một tài nguyên có tối đa 5 diagnostic setting; alert processing rule chặn hoặc thêm action group, không ngăn alert được kích hoạt; metric alert gần thời gian thực, log alert chạy theo chu kỳ query; action group có giới hạn tần suất email/SMS.

Chi phí: ~\$0.3 (VM), log nằm trong 5 GB miễn phí.

Ngày 21 — Insights, Network Watcher, Connection monitor, vault và backup policy

Bám bullet: cấu hình và diễn giải giám sát VM, storage account và mạng bằng Azure Monitor Insights; dùng Network Watcher và Connection monitor; tạo Recovery Services vault; tạo Azure Backup vault; tạo và cấu hình backup policy.

Portal/CLI: bật VM Insights (Azure Monitor Agent + data collection rule) cho VM; mở Storage Insights và Network Insights xem topology; tạo Connection monitor từ VM đến `www.microsoft.com:443`; tạo Recovery Services vault và Backup vault; tạo policy VM backup hằng ngày giữ 7 ngày, bật backup cho VM và chạy "Backup now" để ngày 22 có recovery point.

```

az backup vault create -g rg-az104-mon -n rsv-az104 -l eastus
az backup vault backup-properties set -g rg-az104-mon -n rsv-az104 \
  --backup-storage-redundancy LocallyRedundant --soft-delete-feature-state
Disable # để còn xóa được vault cuối kỳ
az backup protection enable-for-vm -g rg-az104-mon --vault-name rsv-az104 --
vm vm-mon --policy-name DailyPolicy-7d
az backup protection backup-now -g rg-az104-mon --vault-name rsv-az104 --
container-name vm-mon --item-name vm-mon \
  --backup-management-type AzureIaaSVM --retain-until 31-10-2026
az dataprotection backup-vault create -g rg-az104-mon --vault-name bv-az104 -
l eastus --type SystemAssigned \
  --storage-settings datastore-type="VaultStore" type="LocallyRedundant"
az network watcher connection-monitor create -n cm-web -g rg-az104-mon --
location eastus \
  --endpoint-source-resource-id <vmId> --endpoint-source-name vm-mon --
endpoint-dest-address www.microsoft.com \
  --endpoint-dest-name mslearn --test-config-name tcp443 --protocol Tcp --
tcp-port 443

```

Lab C# (Day21): tạo Recovery Services vault bằng SDK, rồi đọc dữ liệu VM Insights và Connection monitor từ workspace.

```

var vault = (await
rg.GetRecoveryServicesVaults().CreateOrUpdateAsync(WaitUntil.Completed, "rsv-
sdk",
    new RecoveryServicesVaultData(AzureLocation.EastUS)
    {
        Sku = new RecoveryServicesSku(RecoveryServicesSkuName.RS0) { Tier =
"Standard" },
        Properties = new RecoveryServicesVaultProperties()
    })).Value;
Console.WriteLine($"Vault {vault.Data.Name}:
{vault.Data.Properties.ProvisioningState}");

var ws = law.Data.CustomerId.ToString();
// VM Insights: % bộ nhớ khả dụng theo VM
var mem = await logs.QueryWorkspaceAsync(ws, ""
    InsightsMetrics
    | where Namespace == "Memory" and Name == "AvailableMB"
    | summarize AvgAvailableMB = avg(Val) by Computer, bin(TimeGenerated,
15m)
    "", new QueryTimeRange(TimeSpan.FromHours(2)));

```

```
// Connection monitor: tỉ lệ check thất bại và độ trễ
var cm = await logs.QueryWorkspaceAsync(ws, ""
    NWConnectionMonitorTestResult
    | summarize Checks = count(), Failed = countif(TestResult == "Fail"),
    AvgRtt = avg(AvgRoundTripTimeMs) by TestGroupName
    "", new QueryTimeRange(TimeSpan.FromHours(2)));
foreach (var row in cm.Value.Table.Rows) Console.WriteLine(string.Join(" | ",
    row));
```

Bắt đề thi: Recovery Services vault dùng cho Azure VM, SQL/SAP HANA trong VM, Azure Files, MARS agent và Site Recovery; Backup vault dùng cho Azure Disks, Azure Blobs, Azure Database for PostgreSQL, AKS; vault phải cùng region với tài nguyên được backup; storage redundancy của vault chỉ đổi được trước khi có item đầu tiên; Connection monitor cần agent trên VM nguồn; VM Insights cần Azure Monitor Agent và data collection rule.

Chi phí: ~\$0.5.

Ngày 22 — Backup/restore, Site Recovery, failover, báo cáo và cảnh báo backup

Bám bullet: backup và restore bằng Azure Backup; cấu hình Azure Site Recovery cho tài nguyên Azure; failover sang region phụ bằng Site Recovery; cấu hình và diễn giải báo cáo, cảnh báo backup.

Thứ tự trong ngày (quan trọng vì replicate mất thời gian):

1. Ngay đầu buổi: bật Disaster recovery cho VM sang region West US (portal, VM → Disaster recovery). Replicate ban đầu chạy nền.
2. Trong lúc chờ: file recovery (mount recovery point, lấy lại một file), rồi restore disks sang resource group khác.
3. Bật diagnostic setting của vault gửi các bảng `AddonAzureBackup*` về workspace, mở Backup reports; tạo alert rule cho job backup thất bại.
4. Khi replicate xong: Test failover vào VNet cách ly, kiểm tra, Cleanup test failover; sau đó Failover thật, Commit, xem VM ở West US; cuối cùng tắt replication.

```

az backup recoverypoint list -g rg-az104-mon --vault-name rsv-az104 --
container-name vm-mon --item-name vm-mon \
  --backup-management-type AzureIaaSVM -o table
az backup restore restore-disks -g rg-az104-mon --vault-name rsv-az104 --
container-name vm-mon --item-name vm-mon \
  --rp-name <rp> --storage-account <st-staging> --target-resource-group rg-
az104-restore
az backup job list -g rg-az104-mon --vault-name rsv-az104 -o table

```

Lab C# (Day22) — BackupReporter: tổng hợp tình trạng job backup và job Site Recovery từ workspace thành báo cáo một dòng mỗi item.

```

var jobs = await logs.QueryWorkspaceAsync(ws, ""
  AddonAzureBackupJobs
  | where JobOperation in ("Backup", "Restore")
  | summarize arg_max(TimeGenerated, JobStatus, JobDurationInSecs,
JobFailureCode) by BackupItemUniqueId, JobOperation
  | project TimeGenerated, JobOperation, BackupItemUniqueId, JobStatus,
DurationMin = JobDurationInSecs / 60, JobFailureCode
  "", new QueryTimeRange(TimeSpan.FromDays(2)));
foreach (var row in jobs.Value.Table.Rows)
  Console.WriteLine($"{row["JobOperation"],-8} {row["JobStatus"],-12}
{row["DurationMin"],5} min {row["JobFailureCode"]}");

var asr = await logs.QueryWorkspaceAsync(ws, ""
  AzureSiteRecoveryJobs
  | project TimeGenerated, OperationName, State =
tostring(column_ifexists("State", "")), AffectedObjectFriendlyName
  | order by TimeGenerated desc
  "", new QueryTimeRange(TimeSpan.FromDays(1)));
foreach (var row in asr.Value.Table.Rows) Console.WriteLine(string.Join(" |
", row));

```

Nếu bảng trống, kiểm tra lại diagnostic setting của vault đã chọn chế độ resource-specific chưa; dữ liệu thường trễ vài chục phút.

Bây đề thi: test failover không ảnh hưởng VM gốc và nên dùng VNet cách ly; sau failover thật phải Commit, muốn quay về thì Re-protect rồi failback; Site Recovery miễn phí 31 ngày đầu cho mỗi instance; file recovery mount recovery point dưới dạng ổ đĩa qua script; restore có ba kiểu: tạo VM mới, restore disks, thay thế disk của VM hiện có; soft delete giữ dữ liệu backup đã xóa 14 ngày và chặn việc xóa vault; Backup reports dựa trên dữ liệu diagnostic trong Log Analytics.

Chi phí: ~\$2-3. Cuối ngày: dừng backup và xoá dữ liệu backup, tắt replication, rồi mới xoá `rg-az104-mon`, `rg-az104-restore` và resource group Site Recovery tạo ở West US.

Ngày 23-24: Practice assessment và sửa lỗi hỏng

Hai ngày cuối không học nội dung mới. Mục tiêu là biến danh sách câu sai thành lab làm lại, vì với AZ-104, câu sai thường do chưa tự tay làm chứ không phải do chưa đọc.

Ngày 23 — Practice assessment lần 1 và lab bù

1. Làm [Practice Assessment miễn phí](#) trên Microsoft Learn, không tra cứu.
2. Ghi mỗi câu sai vào bảng có cột "gạch đầu dòng" (tra bảng ở Phụ lục) và "vì sao sai": chưa biết, nhầm hai dịch vụ, hay đọc sót điều kiện.
3. Chọn 3 gạch đầu dòng sai nhiều nhất, làm lại đúng lab của ngày đó bằng portal và CLI, 30 phút mỗi bài. Phần credit còn dư dùng cho bước này.
4. Mở [exam sandbox](#) để quen dạng case study và drag-and-drop.

Chi phí: ~\$2-5 tùy lab làm lại.

Ngày 24 — Practice assessment lần 2, luyện tra cứu, dọn dẹp

1. Làm lại practice assessment (bộ câu hỏi xáo trộn), so điểm với ngày 23.
2. Đọc lại toàn bộ các mục "Bẫy đề thi" trong tài liệu này, tập trung vào các cặp dễ nhầm: service endpoint và private endpoint, Recovery Services vault và Backup vault, Incremental và Complete, scale up và scale out, NSG subnet và NSG NIC.
3. Luyện tra nhanh Microsoft Learn (được mở trong lúc thi các exam role-based) cho các bảng hay cần: redundancy storage, tính năng theo tier App Service, giới hạn VNet/NSG, SKU Bastion.
4. Chạy script dọn dẹp ở Phụ lục, kiểm tra không còn resource group gắn tag `course=az104`, không còn vault hay replication nào ở West US.

Đặt lịch thi trong khoảng ngày 25-28, trước khi credit hết hạn ở ngày 30.

Phụ lục

Ảnh xạ 82 gạch đầu dòng sang ngày học

Ngày	Gạch đầu dòng trong study guide (tóm tắt)	Số
1	Tạo user và group; quản lý thuộc tính user và group	2
2	License Entra ID; external user; SSPR	3
3	Built-in role; gán role ở nhiều scope; diễn giải access assignment	3
4	Azure Policy; resource lock; tag	3
5	Resource group; subscription; chi phí (alert, budget, Advisor); management group	4
6	Tạo/cấu hình storage account; redundancy; encryption	3
7	Firewall và VNet cho storage; SAS; stored access policy; access key	4
8	Blob container; storage tier; soft delete blob/container; lifecycle; versioning; object replication; Storage Explorer và AzCopy	7
9	File share; snapshot và soft delete Azure Files; identity-based access cho Azure Files	3
10	Diễn giải ARM/Bicep; sửa ARM; sửa Bicep; deploy; export/decompile	5
11	Tạo VM; encryption at host; VM size; VM disk	4
12	Availability zone/set; di chuyển VM; VM Scale Sets	3
13	ACR; ACI; Container Apps; sizing và scaling container	4
14	App Service plan; scaling plan; tạo App Service; backup; deployment slot	5
15	Certificate và TLS; custom DNS name; networking App Service	3

Ngày	Gạch đầu dòng trong study guide (tóm tắt)	Số
16	VNet và subnet; peering; public IP; user-defined route	4
17	NSG và ASG; effective security rules; Azure Bastion	3
18	Service endpoint; private endpoint; Azure DNS	3
19	Internal/public load balancer; troubleshoot load balancing; troubleshoot kết nối mạng	3
20	Metric; log settings; query log; alert rule, action group, alert processing rule	4
21	Monitor Insights (VM, storage, network); Network Watcher và Connection monitor; Recovery Services vault; Backup vault; backup policy	5
22	Backup và restore; cấu hình Site Recovery; failover sang region phụ; báo cáo và cảnh báo backup	4

Script dọn dẹp cuối ngày

```
# Giữ các group nhiều ngày khi đang trong chạng đó, bỏ tên khỏi KEEP khi qua chạng
KEEP="rg-az104-net rg-az104-mon"
for g in $(az group list --query "[?tags.course=='az104'].name" -o tsv); do
  [[ " $KEEP " == "$g" ]] && continue
  for l in $(az lock list -g $g --query "[].id" -o tsv); do az lock delete --ids $l; done # lock chặn xóa (ngày 4)
  az group delete -n $g --yes --no-wait && echo "deleting $g"
done
az group list --query "[?tags.course=='az104'].{name:name, state:properties.provisioningState}" -o table
```

Nhớ gắn `--tags course=az104` cho mọi resource group tạo bằng CLI (hàm `Az.RgAsync` trong C# đã tự gắn). Resource group do dịch vụ tự sinh như `NetworkWatcherRG` hay group của Site Recovery ở West US không có tag này, nên kiểm tra tay ở ngày 24.

Nguồn

- [Study guide for Exam AZ-104: Microsoft Azure Administrator](#), Microsoft Learn, skills measured từ 17/04/2026, trang cập nhật 19/03/2026.